EECS 563 Homework 2 - Solution

Due: Friday, September 18, 2026, 11:59 pm. Online submission on Canvas.

Problem 1 [8 points]: Consider sending a large amount of data over a lossy link. Each bit can be independently in error with a probability of P. Each packet consists of two parts: ***data*** and ***header*** parts. Packets with any bit (data or header) in error are dropped by the receiver. Each packet contains H bit headers and D bits data. The raw capacity of the link is C bits per second.

(a) What is the probability that a packet gets dropped at the receiver?

Solution: A packet survives only if all $H + D$ bits are correct, each with probability $(1 - P)$. Thus

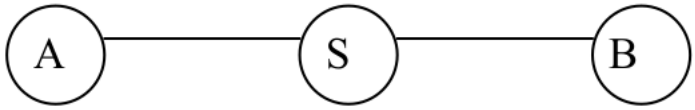
$$P_{\text{drop}} = 1 - (1 - P)^{H+D}.$$

(b) Write the expression for the throughput over the link. Note that: Throughput is defined as the maximum number of ***data bits*** received ***successfully*** per second by the application. Assume that the sender is sending packets back-to-back.

Solution: Back-to-back sending gives a packet rate of $\frac{C}{H+D}$ packets/sec. Each delivered packet carries D useful bits, and a packet is received successfully with probability $(1 - P)^{H+D}$. Hence

$$\text{Throughput} = \frac{C}{H + D} \cdot D \cdot (1 - P)^{H+D} = C \, \frac{D}{H + D} \, (1 - P)^{H+D} \quad \text{bits/sec.}$$

Problem 2 [12 points]: Consider the network depicted in the figure below where nodes A and B are hosts, and node S is a store-and-forward switch. The switch has a very large buffer and never loses packets.



Suppose that link A→ S has a bandwidth of 2 Mb/s and propagation delay of 2 ms, and link S→ B has a bandwidth of 1 Mb/s and propagation delay of 1 ms.

(a) Assume that node A has a data message of size 8,000 bits that it wishes to transmit to node B. What is the end-to-end delay if node A sends the ***entire message as a single packet***?

Solution: On the link from A to S a packet of size 8,000 bits will take 4 ms to

transmit. With the propagation delay, the last bit of the packet will arrive 6 ms after packet transmission starts at A. Since S operates in the store-and-forward fashion it will start transmission after the entire packet is received, and transmission on the S to B link will take 8ms + 1 ms = 9 ms. Hence, the end-to-end delay in this case is 15ms.

(b) Now, assume that node A divides data into **4 *equal-sized packets*** (i.e., each packet has $2,000$ bits). In this case, what is the end-to-end delay between A and B? Assume a packet enters the switch as soon as the first bit arrives, and leaves the switch only when the last bit is transmitted. Assume that the packets are sent back-to-back.

Solution: In this case, packet size will be $8,000/4 = 2000$ bits. Therefore, transmission delay for each packets will be smaller. Moreover, switch S can start sending the first packet as soon as it receives all the bits of the first packets (i.e., 2000 bits). Compare this with Part (a) where switch S needed to received all 8,000 bits (all bits of the packet) before being able to send bits to B.

You can repeat the same calculation as Part a (for smaller packets). Then, you obtain that:

- 1st packet arrives in B at 6 ms
- 2nd packet arrives in B at 8 ms
- 3rd packed arrives in B at 10 ms
- 4th packet arrives in B at 12 ms

Therefore, the end to end delay is now 12 ms. Splitting the message into smaller packets is called **Segmentation.** By segmentation, we can reduce the end to end delay. However, we introduce other problems, namely:

- Less efficient communications: ratio of useful bits to header bits within each packet decreases
- Possibility of reorder delivery

Problem 3 [6 points]: HTTP Messages.

(a) A user requests a web page consisting of a base object and three referenced images. Determine the total number of HTTP requests and HTTP response messages exchanged. Also specify the number of TCP connections established when using non-persistent HTTP and persistent HTTP. Assume that round trip trip is denoted by T. How much time is saved by using persistent HTTP compared to non-persistent HTTP? Justify your answer.

Solution: There are 4 objects total (1 base object + 3 images). Hence, the total number of HTTP request messages is 4, and the total number of HTTP response messages is also 4.

With **non-persistent HTTP**, there is one TCP connection per object, so the number of TCP connections is 4. Each object requires $1T$ to establish the TCP connection plus $1T$ for the HTTP request/response, so the total time is

$$4(2T) = 8T.$$

With **persistent HTTP** (single TCP connection reused for all objects), the number of TCP connections is 1. It takes $1T$ to establish the TCP connection, plus $1T$ to request/receive the base object, plus $3T$ to request/receive the three images (one RTT each, sent sequentially), giving total time

$$1T + 1T + 3T = 5T.$$

Therefore, the time saved by using persistent HTTP compared to non-persistent HTTP is

$$8T - 5T = 3T.$$

(b) Can an HTTP response message have an empty message body? Justify your answer.

Solution: HTTP uses caching to save bandwidth. Therefore, it is possible for the message body to be empty if the client has a copy of the object that has not been modified. Another instance is when the HTTP request message uses the HEAD method, which requests headers (only) from the HTTP server.

Problem 4 [6 points]: Wireshark is a packet sniffer, a software program that receives the packets going out of and into your computer. Packet sniffer does not actively participate in the network, it does not send out its own packets, but merely captures the packets as they are transmitted. By now, you should have installed Wireshark on the Kali linux using the instruction provided earlier. Open a terminal and type Wireshark to open it.

Follow these steps:

- From the main Wireshark screen, you can select the interface (e.g., eth0) over which you want to capture packets.
- Start capturing packets over the selected interface.
- Open Mozilla Firefox, the default browser installed on Kali Linux, which will display your selected homepage.
- Open few websites of your choice on the browser.
- Stop the packet capture after a few minutes.

Now, answer the following questions. **Please include screenshots of Wireshark terminal to support your answers.**

(a) List 3 different network protocols that appear in the protocol column in the Wireshark window.
Solution: [3 points] In this exercise, we browsed ku.edu, wireshark.org, youtube.com. The protocols seen in wireshark window include QUIC, TCP, DNS, TLSv1.3, as shown in Fig. 1.

(b) What are the Internet addresses (i.e., IP address) of your host and at least for two websites you visited?
Solution: [3 points] The IPv4 address of the host in the above example is 192.168.160.128. In this example, the IP address of ku.edu and wireshark.org are 129.237.135.76, 104.26.10.240 respectively. Your answers vary depending on the websites you visited.

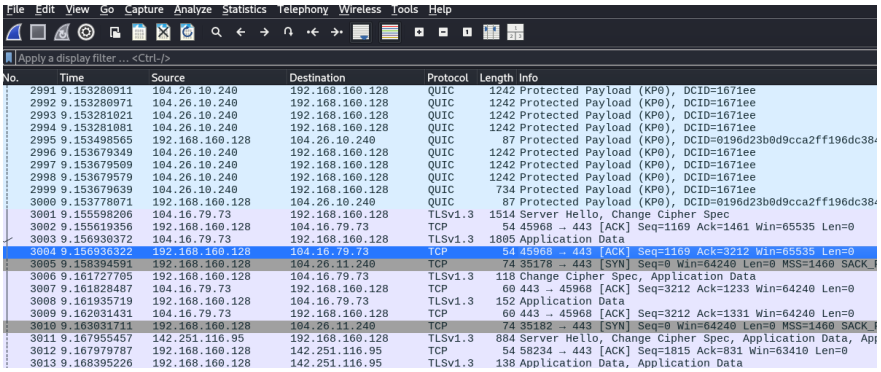


Figure 1: An Example of Wireshark Capture

Problem 5 [12 points]: Repeat the steps above to open Wireshark for packet capture. In this experiment, you will get familiar with the basic HTTP GET/response interaction, and capturing error codes. Follow these steps, and answer the questions. **Please include screenshots of Wireshark terminal to support your answers.**

- Start capturing packets over the selected interface.
- Open Mozilla Firefox, and visit a website that uses http. For example visit, `http://gaia.cs.umass.edu/wireshark-labs/TCP-wireshark-file1.html`
- Download the text file alice.txt (from the same link) and upload it to the website.
- Stop the packet capture after a few minutes.

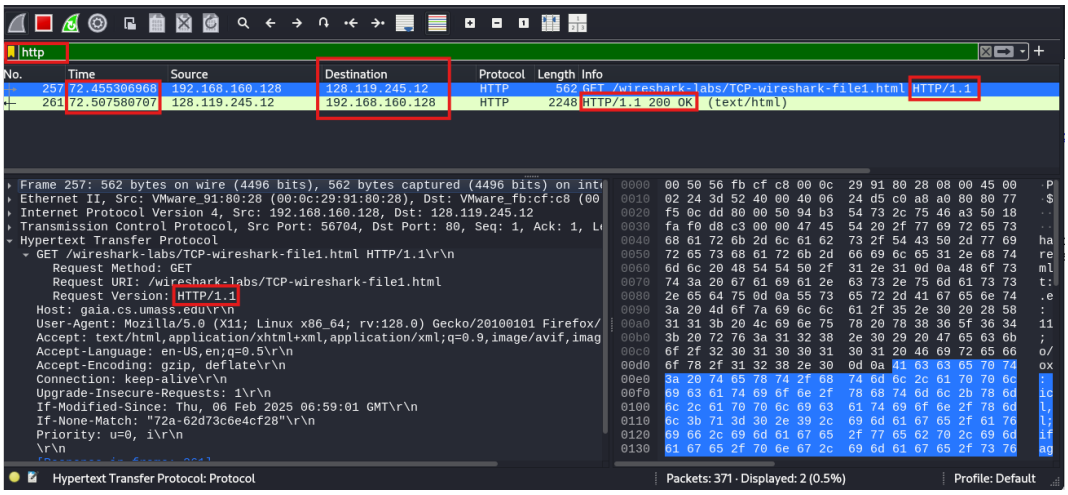


Figure 2: Wireshark Capture for Problem 5

(a) Type in `http` (lower case) into the display filter specification window at the top of the main Wireshark window. Then select Apply. This will cause only HTTP message to be displayed in the packet-listing window. From the time column, calculate how long it took from when the HTTP GET message was sent until the HTTP OK reply was received.

Solution: [2 points] As shown in Fig. 2, HTTP request is sent in packet 257, and the response is received is packet 261. The time taken to complete request and response is 72.5075 – 72.4553 = 0.0522 seconds.

(b) Is your browser running HTTP version 1.0 or 1.1? What version of HTTP is the server running?

Solution: [2 points] As shown in Fig. 2, both browser and the server are using HTTP version 1.1.

(c) What are the Internet addresses (i.e., IP address) of your host and web server?

Solution: [2 points] The IP address of the host is 192.168.160.128, and the IP address of the server is 128.119.245.12.

(d) What is the status code returned from the server to your browser?

Solution: [2 points] The status code observed is 200.

(e) Repeat the same experiment, Start Wireshark capture again, open the same website `http://gaia.cs.umass.edu/wireshark-labs/TCP-wireshark-file2.html`. Note that the file name is changed to a name that does not exist on the HTTP server. This should throw a "Not Found" error. Are you able to capture this error in Wireshark? What is the error code you observed?

Solution: [2 points] For the no found error, we observe error code as 404, as shown in Fig. 3.

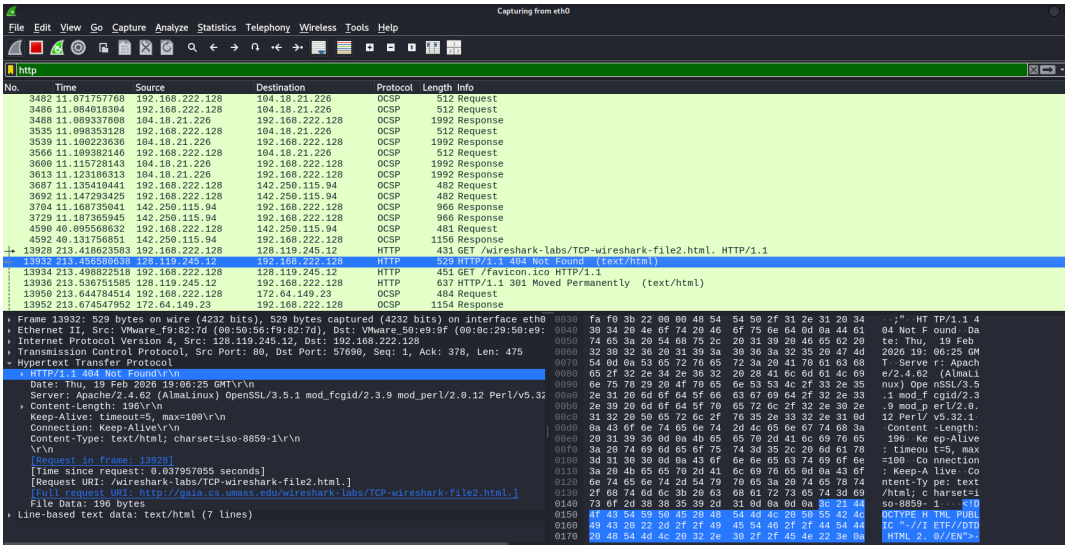


Figure 3: Wireshark Capture for 404 error code.

(f) Repeat the above experiment on a https website, for example: visit `https://eecs.ku.edu`. What protocol is used for the request and response when accessing the website?

Solution: [2 points] For https websites, firefox on Kali uses Online Certificate Status Protocol (OCSP).